

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.metrics import silhouette_score
```

```
df=pd.read_excel("Online_Retail.xlsx")
```

```
df
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-01-12 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-01-12 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-01-12 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-01-12 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-01-12 08:26:00	3.39	17850.0	United Kingdom
...	...	...	...	...	...	...	...	...
541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	2011-09-12 12:50:00	0.85	12680.0	France
541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	2011-09-12 12:50:00	2.10	12680.0	France

```
print(df.head())
```

```

InvoiceNo  StockCode  Description  Quantity \
0  536365  85123A  WHITE HANGING HEART T-LIGHT HOLDER  6
1  536365  71053  WHITE METAL LANTERN  6
2  536365  84406B  CREAM CUPID HEARTS COAT HANGER  8
3  536365  84029G  KNITTED UNION FLAG HOT WATER BOTTLE  6
4  536365  84029E  RED WOOLLY HOTTIE WHITE HEART.  6

InvoiceDate  UnitPrice  CustomerID  Country
0  2010-01-12 08:26:00  2.55  17850.0  United Kingdom
1  2010-01-12 08:26:00  3.39  17850.0  United Kingdom
2  2010-01-12 08:26:00  2.75  17850.0  United Kingdom
3  2010-01-12 08:26:00  3.39  17850.0  United Kingdom
4  2010-01-12 08:26:00  3.39  17850.0  United Kingdom
```

```
# Missing Values
```

```
df.isnull().sum()
```

```

InvoiceNo    0
StockCode    0
Description  1454
Quantity     0
InvoiceDate  0
UnitPrice    0
CustomerID  135080
Country      0
```

```
dtype: int64
```

```
df=df.dropna(subset=['CustomerID'])
```

```
# Remove cancelled orders

cancelled_orders=df['InvoiceNo'].astype(str).str.startswith('C').sum()

print("Number of cancelled orders:",cancelled_orders)
```

Number of cancelled orders: 0

```
df = df[
    ~df['InvoiceNo']
    .astype(str)
    .str.startswith('C')
]
```

```
# Check and Remove Negative Quantity

print((df['Quantity']<=0).sum())
```

0

```
df=df[df['Quantity']>0]
```

```
print((df['Quantity']<=0).sum())
```

0

```
# Check and remove Negative Unit Price

print((df['UnitPrice']<=0).sum())
```

0

```
df=df[df['UnitPrice']>0]
```

```
print((df['UnitPrice']<=0).sum())
```

0

Feature Engineering

```
# Total Amount

df['TotalAmount']=df['Quantity']*df['UnitPrice']
```

```
df['TotalAmount']
```

	TotalAmount
0	15.30
1	20.34
2	22.00
3	20.34
4	20.34
...	...
541904	10.20
541905	12.60
541906	16.60
541907	16.60
541908	14.85

397884 rows × 1 columns

dtype: float64

```
# Date conversion
```

```
df['InvoiceDate']=pd.to_datetime(df['InvoiceDate'])
```

Create RFM Features

```
# Recency

snapshot_date = (df['InvoiceDate'].max()
                 + pd.Timedelta(days=1))

rfm = (df.groupby('CustomerID')['InvoiceDate'].max().reset_index())

rfm['Recency'] = (snapshot_date - rfm['InvoiceDate']).dt.days
```

```
# Frequency

frequency = (df.groupby('CustomerID')['InvoiceNo'].nunique().reset_index())

frequency.columns=['CustomerID', 'Frequency']
```

```
# Monetary

monetary = (df.groupby('CustomerID')['TotalAmount'].sum().reset_index())

monetary.columns=['CustomerID', 'Monetary']
```

```
# Merge RFM

rfm = rfm[['CustomerID', 'Recency']]

rfm = rfm.merge(frequency[['CustomerID', 'Frequency']], on='CustomerID')

rfm = rfm.merge(monetary[['CustomerID', 'Monetary']], on='CustomerID')
```

```
print(rfm.columns)
```

```
Index(['CustomerID', 'Recency', 'Frequency', 'Monetary'], dtype='object')
```

DATA EXPLORATION

```
# How many total customers are present in the dataset?

total_customers=df['CustomerID'].nunique()

print("Total customers present are:",total_customers)
```

```
Total customers present are: 4338
```

```
# How many features are used for clustering after preprocessing?

X = rfm[
    ['Recency', 'Frequency', 'Monetary']
]
print("Number of features:",X.shape[1])
```

```
Number of features: 3
```

```
# Which feature has the highest variance among all features?

variance = X.var()

print(variance)

print("Feature with highest variance:",variance.idxmax())
```

```
Recency      1.324390e+04
Frequency    5.925917e+01
Monetary     8.080626e+07
dtype: float64
Feature with highest variance: Monetary
```

```
X = rfm[['Recency', 'Frequency', 'Monetary']]
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# Elbow Method

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

wcss = []

for i in range(1,11):

    kmeans = KMeans(n_clusters=i, random_state=0)

    kmeans.fit(X_scaled)

    wcss.append(kmeans.inertia_)
```

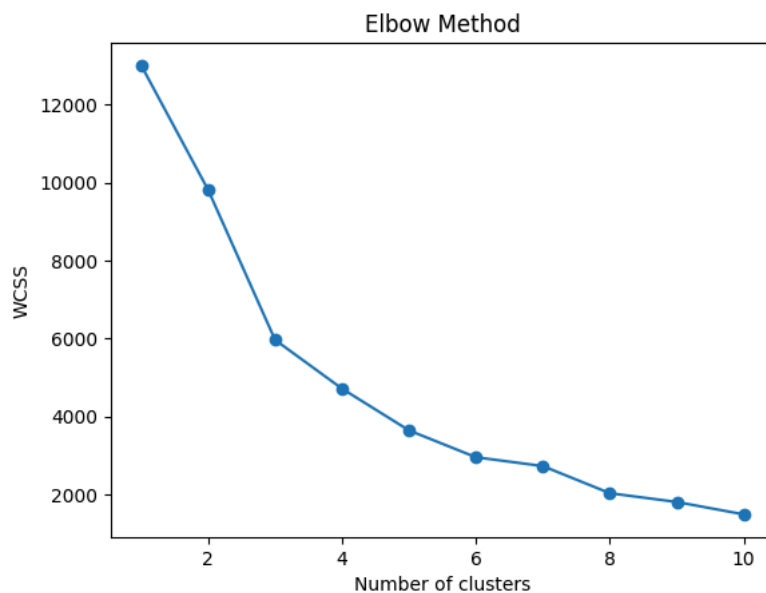
```
plt.plot(range(1,11),wcss,marker='o')

plt.title('Elbow Method')

plt.xlabel('Number of clusters')

plt.ylabel('WCSS')

plt.show()
```



```
# K-Means

kmeans = KMeans(n_clusters=4, random_state=0)

rfm['Cluster'] = kmeans.fit_predict(X_scaled)
```

```
rfm.head()
```

	CustomerID	Recency	Frequency	Monetary	Cluster	Distance
0	12346.0	327	1	77183.60	0	8.542974
1	12347.0	41	7	4310.00	1	0.401661
2	12348.0	77	4	1797.24	1	0.264098
3	12349.0	20	1	1757.55	1	0.548027
4	12350.0	312	1	334.40	0	0.442671

CLUSTER FORMATION

```
# How many clusters were finally created by the clustering algorithm?

rfm['Cluster'].nunique()
```

4

```
# What is the size (number of customers) of each cluster?

cluster_size = rfm['Cluster'].value_counts().sort_index()

print(cluster_size)
```

```
Cluster
0    1178
1    3132
2         4
3        24
Name: count, dtype: int64
```

```
# Which cluster contains the highest number of customers?

highest_cluster = rfm['Cluster'].value_counts().idxmax()

print("Cluster with highest number of customers are:", highest_cluster)

highest_size = rfm['Cluster'].value_counts().max()

print("Size of the cluster are:", highest_size)
```

```
Cluster with highest number of customers are: 1
Size of the cluster are: 3132
```

CLUSTER CHARACTERISTICS

```
# What is the average rfm values of customers in each cluster?

cluster_summary = (rfm.groupby('Cluster').mean())

print(cluster_summary)
```

	CustomerID	Recency	Frequency	Monetary	Distance
Cluster					
0	15309.384550	261.921053	1.551783	569.484517	0.664652
1	15298.124202	48.825670	4.749361	1908.882102	0.617809
2	16661.000000	27.500000	45.250000	225721.652500	5.819588
3	14931.083333	12.333333	68.666667	56627.074583	5.926337

```
# Which cluster has the highest average spending score?

highest_spending_cluster = (rfm.groupby('Cluster')['Monetary'].mean().idxmax())

print("Cluster with highest average spending score are:", highest_spending_cluster)
```

```
Cluster with highest average spending score are: 2
```

```
# Which cluster represents the lowest spending customers?

lowest_spending_cluster = (rfm.groupby('Cluster')['Monetary'].mean().idxmin())

print("Cluster with lowest average spending score are:", lowest_spending_cluster)
```

```
Cluster with lowest average spending score are: 0
```

MODEL EVALUATION

```
# What is the inertia value obtained from the K-Means clustering model?

print("inertia:", kmeans.inertia_)
```

```
inertia: 4713.1490018209415
```

```
# What is the silhouette score of the clustering result?

print(silhouette_score(X_scaled, rfm['Cluster']))
```

```
0.5475310517354021
```

```
# Based on the elbow method, what is the optimal number of clusters?

print("Optimal number of clusters K are:", kmeans.n_clusters)
```

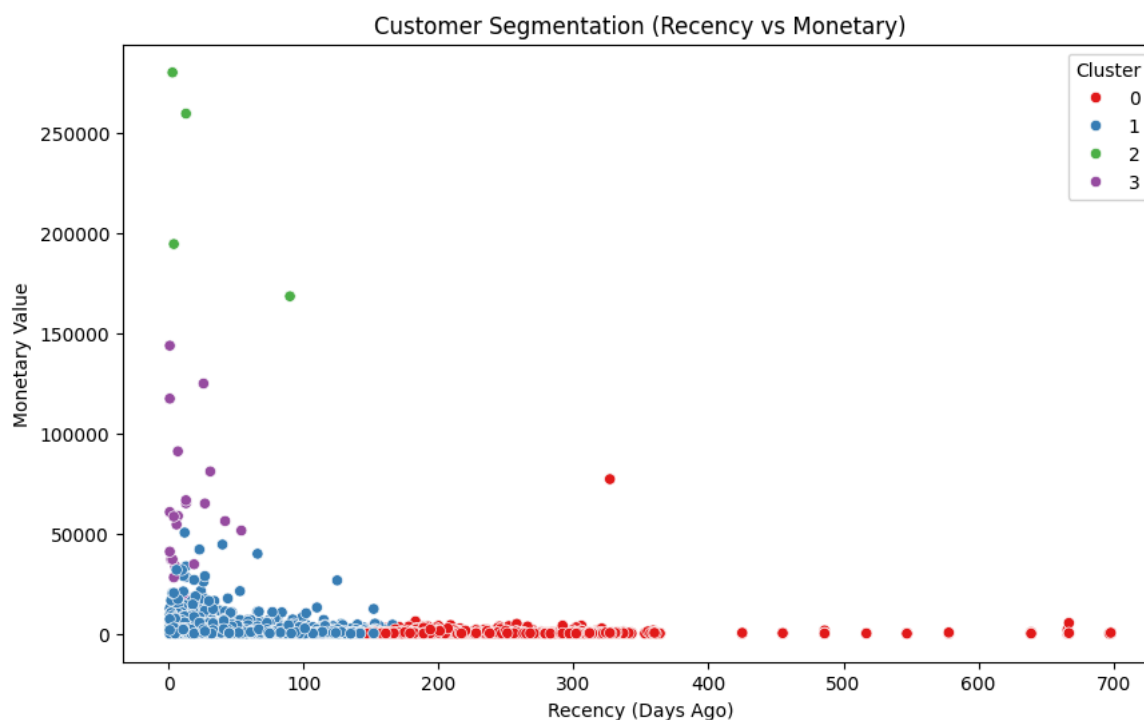
Optimal number of clusters K are: 4

VISUALIZATION-BASED QUESTIONS

```
plt.figure(figsize=(10,6))

sns.scatterplot(
    x='Recency',
    y='Monetary',
    hue='Cluster',
    palette='Set1',
    data=rfm
)

plt.title("Customer Segmentation (Recency vs Monetary)")
plt.xlabel("Recency (Days Ago)")
plt.ylabel("Monetary Value")
plt.show()
```



In the cluster visualization plot, which cluster appears most densely populated?

```
densest_cluster = rfm['Cluster'].value_counts().idxmax()

print("Most Densely Populated Cluster:", densest_cluster)
```

Most Densely Populated Cluster: 1

Which cluster represents customers with high income but low spending score?

```
cluster_summary = rfm.groupby('Cluster')[
    ['Recency', 'Frequency', 'Monetary']
].mean()

print(cluster_summary)
```

Cluster	Recency	Frequency	Monetary
0	261.921053	1.551783	569.484517
1	48.825670	4.749361	1908.882102
2	27.500000	45.250000	225721.652500
3	12.333333	68.666667	56627.074583

```
high_value_cluster = (
    cluster_summary['Monetary']
    .idxmax()
)

print("High Value Customer Cluster:", high_value_cluster)
```

High Value Customer Cluster: 2

```
# Which cluster represents low income but high spending customers?
```

```
cluster_summary = rfm.groupby('Cluster')[
    ['Recency', 'Frequency', 'Monetary']
].mean()

print(cluster_summary)
```

Cluster	Recency	Frequency	Monetary
0	261.921053	1.551783	569.484517
1	48.825670	4.749361	1908.882102
2	27.500000	45.250000	225721.652500
3	12.333333	68.666667	56627.074583

```
low_value_cluster = (
    cluster_summary['Monetary']
    .idxmin()
)

print("Low Value Customer Cluster:", low_value_cluster)
```

Low Value Customer Cluster: 0

CLUSTER LABEL ANALYSIS

```
cluster_summary = rfm.groupby('Cluster')[
    ['Recency', 'Frequency', 'Monetary']
].mean()

print(cluster_summary)
```

Cluster	Recency	Frequency	Monetary
0	261.921053	1.551783	569.484517
1	48.825670	4.749361	1908.882102
2	27.500000	45.250000	225721.652500
3	12.333333	68.666667	56627.074583

```
# Which cluster would most likely represent premium customers?
```

```
print("Premium customer cluster:", cluster_summary['Monetary'].idxmax())
```

Premium customer cluster: 2

```
# Which cluster might correspond to price-sensitive customers?
```

```
print("Price-sensitive customer cluster:", cluster_summary['Monetary'].idxmin())
```

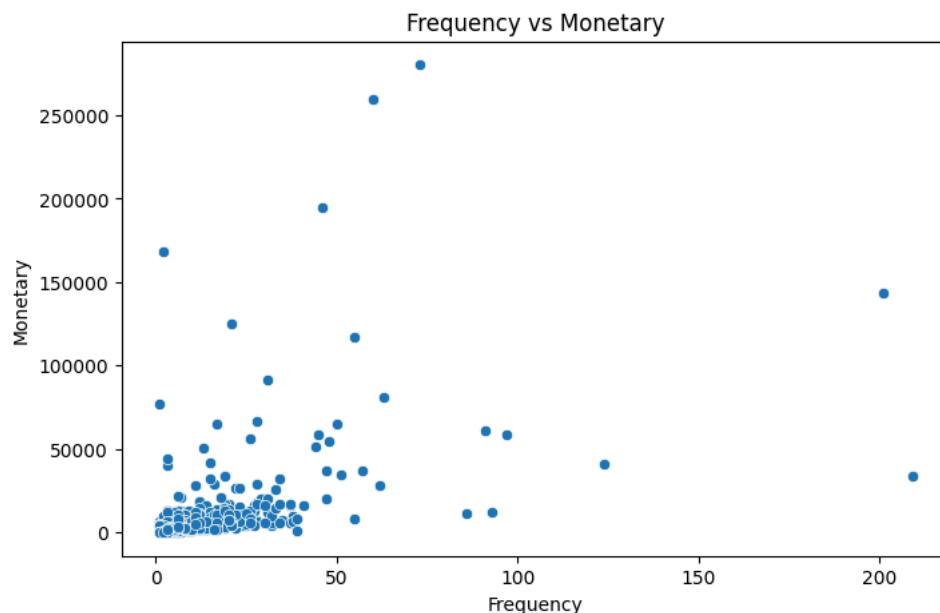
Price-sensitive customer cluster: 0

OUTLIER DETECTION

```
plt.figure(figsize=(8,5))

sns.scatterplot(
    x='Frequency',
    y='Monetary',
    data=rfm
)

plt.title('Frequency vs Monetary')
plt.show()
```



```
# Are there any potential outliers visible in the dataset after clustering?
```

```
rfm['Distance'] = np.linalg.norm(
    X_scaled - kmeans.cluster_centers_[rfm['Cluster']],
    axis=1
)
```

```
len(rfm.nlargest(10, 'Distance'))
```

```
10
```

```
rfm['Distance'].describe()
```

	Distance
count	4338.000000
mean	0.664695
std	0.803000
min	0.037703
25%	0.353088
50%	0.514837
75%	0.699893
max	19.741128

```
dtype: float64
```

```
# Which data points appear farthest from cluster centroids?
```

```
outliers = rfm.nlargest(10, 'Distance')
```

```
print(outliers)
```

	CustomerID	Recency	Frequency	Monetary	Cluster	Distance
1879	14911.0	1	201	143825.06	3	19.741128
326	12748.0	5	209	33719.73	3	18.409330
55	12415.0	26	21	124914.53	3	9.802362
0	12346.0	327	1	77183.60	0	8.542974
3008	16446.0	90	2	168472.50	2	8.510992
4010	17841.0	1	124	40991.57	3	7.396976
1689	14646.0	3	73	280206.02	2	7.056076
1333	14156.0	1	55	117379.63	3	6.989164
1289	14096.0	13	17	65164.79	3	6.779354
3771	17511.0	7	31	91062.38	3	6.215098

CENTROID ANALYSIS

```
# What are the centroid values for each cluster?

centroids = pd.DataFrame(
    scaler.inverse_transform(kmeans.cluster_centers_),
    columns=['Recency', 'Frequency', 'Monetary']
)

print(centroids)
```

	Recency	Frequency	Monetary
0	261.823579	1.551315	569.355506
1	48.794315	4.750559	1909.358468
2	27.500000	45.250000	225721.652500
3	12.333333	68.666667	56627.074583

```
# Which cluster centroid has the highest average income value?

centroids = pd.DataFrame(
    scaler.inverse_transform(kmeans.cluster_centers_),
    columns=['Recency', 'Frequency', 'Monetary']
)

print(
    "Cluster with Highest Monetary Value:",
    centroids['Monetary'].idxmax()
)
```

Cluster with Highest Monetary Value: 2